Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Mediator kalıbı (ekrandan, 00:12:05, 00:15:40, 00:19:10, 00:20:00)
  - Davranışsal kalıpların sonuncusu olarak işlendi. Kelime anlamı "ara bulucu"dur; hoca bunu iki taraf arasında uzlaşma arayan "negotiator"dan daha geniş ölçekli bir uzlaştırıcı olarak tarif etti.
  - Amaç: bir nesne topluluğu arasındaki etkileşimi tek bir nesnede sarmalamak. Nesneler birbiriyle doğrudan değil, Mediator üzerinden dolaylı iletişim kurar. Nesneler değiştikçe aralarındaki ilişkilerin de değişmesi gerekmez; olası değişiklikler ortadaki Mediator'da sınırlı kalır, diğer nesnelere yayılmaz.
  - Dörtlü Çete'nin örneği: font seçme penceresi (Font Chooser). Family listesi, Weight (medium, bold, demibold) ve Slant (roman, italic, oblique) radyo düğmeleri, Size alanı vardır. Herhangi bir ayardaki değişiklik en üstteki örnek yazıya ("The quick brown fox…") hemen yansır. Bazı bileşenler birbirini dışlayabilir (aynı anda hem medium hem bold seçilemez).
  - Nesne diyagramı: `aClient`, `aListBox`, `aButton`, `anEntryField` her biri `director` referansıyla ortadaki `aFontDialogDirector`'a bağlıdır; director da bunlara geri bağlıdır.
  - Örnek çözüm: GUI nesnelerine ortak bir arayüz (`Widget`, `Changed()`) hazırlanır; aralarındaki etkileşimleri genel olarak tanımlayan soyut bir sınıf (`DialogDirector`: `ShowDialog()`, soyut `CreateWidgets()`, soyut `WidgetChanged(Widget)`) tasarlanır. `Widget.Changed()` içinde `director->WidgetChanged(this)` çağrılır. Somut `FontDialogDirector` (`CreateWidgets()`, `WidgetChanged(Widget)`), `list` ile `ListBox`'a (`GetSelection()`), `field` ile `EntryField`'a (`SetText()`) bağlıdır. Bu soyut sınıftan gerçekleme oluşturmak için Template Method kalıbı kullanılır.
  - Üstteki soyut düzeyde ilişki Widget'tan Director'a doğrudur; alttaki somut düzeyde ise somut Director'dan somut widget'lara doğru ilişki vardır, çünkü somut director onlardan bilgi almak ve onlara iş yaptırmak zorundadır.
  - Kalıp yapısı: `Colleague`, `mediator` referansıyla soyut `Mediator`'a bağlıdır; `ConcreteMediator`, `ConcreteColleague1` ve `ConcreteColleague2`'ye bağlıdır.
  - Bileşenler: `Mediator`, Colleague nesneleriyle etkileşim kurabilecek bir arayüz sağlar. `ConcreteMediator`, Colleague nesneleri arasında eşgüdüm sağlar ve bunların idamesinden sorumludur (sahiplik ilişkisi). `Colleague`/`ConcreteColleague`, etkileşimde bulunacak nesnelerin arayüzü ve gerçeklemeleridir.
  - "Colleague" terimi "meslektaş" demektir. Hoca "peer" yerine bunun seçilmesini yerinde buldu: uzmanlık alanları farklı olabilen ama bir iş için birlikte çalışan nesneler.
  - Mediator'dan bütün Colleague'lere ilişki olması şart değildir. Somut Mediator'ın hiçbir iş yaptırmayacağı bir Colleague varsa o yönde ilişki gerekmez.
- Mediator ve Observer farkı
  - İlk bakışta Mediator gözlemci, Colleague'ler gözlenen nesneler gibi görünebilir; ama durum böyle değildir. Observer'da amaç bir nesnenin durum değişikliğinden haberdar olmaktır. Mediator'da ise eş bileşenler arasındaki karmaşık M×N ilişki tek bir merkezi noktaya toplanır ve herhangi bir bileşen herhangi bir anda bir işin yapılmasını, bir eylemler zincirinin tetiklenmesini isteyebilir.
- Mediator'ın kazancı ve bedeli
  - Beş Colleague birbiriyle tam bağlı (fully connected) etkileşirse kendisiyle etkileşim sayılmadan $5 \times 4 = 20$ ilişki olur. Mediator konulunca bütün oklar çift yönlü sayılsa bile 10 ilişki kalır (ekrandan, 00:20:00).

  - Bedel: bütün merkezi sistemlerde olduğu gibi `ConcreteMediator`'daki bir hata diğer işlerin yürütülmesini aksatabilir. Mediator gerçeklemesi aşırı karmaşık olur; bu yüzden başka projede yeniden kullanımı genelde kolay değildir. Nesneye yönelim "böl ve yönet" mantığını önerir, böyle iri sınıfları sevmez. Bu bir ödünleşimdir (trade-off): nesneler arası bireysel ilişkiler fazla karmaşıksa, merkezi noktadaki kod karmaşıklığı pahasına Mediator uygulanır.
  - Hoca bu kalıp için ayrı bir kod örneği yapmadı; temel bir fikir olarak verdi.
- Refactoring'e giriş (ekrandan, 00:26:30, 00:30:00)
  - Refactoring Türkçede "yeniden yapılandırma" diye çevrilebilir, ama oturmuş bir karşılığı olmadığı için derste "refactoring" denecek. Hoca çeviriyi finans dilindeki "borçların yeniden yapılandırılması"ndan aldığını söyledi.
  - Tanım: bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmek; yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi.
  - Amaç: mevcut kod üzerinde kodun iç kalite özelliklerini iyiye götürecek değişiklikler. Slayttaki maddeler: anlaşılabilirliğin artırılması, esneklik kazandırılması, yeniden kullanılabilirliğin artırılması, yüksek uyum ve düşük bağımlılığın sağlanması. Bu sırada public metot imzaları ve arayüzler değiştirilmemeye çalışılır.
  - İç ve dış kalite özellikleri: iç kalite özellikleri yazılımı geliştirenleri, dış kalite özellikleri yazılımın kullanıcılarını ilgilendirir. Refactoring iç kaliteyi hedefler.
  - Açıklayıcı olmayan bir metot ya da üye alan adını daha açıklayıcı yapmak bile bir refactoring eylemidir.
- Refactoring nasıl ve ne zaman yapılır? (ekrandan, 00:30:00)
  - Çevik yaklaşımla yapılır: mevcut gereksinimleri karşılayacak ilk tasarım yapılır; ilk tasarımın mükemmel olması ve her bilinmezi öngörmesi gerekmez; yeterince iyi tasarım gerçeklenmeye başlanır; önceki tasarım adımlarından birinde yanlış karar verildiği anlaşılınca refactoring yapılır.
  - Küçük artımlarla ilerlenir; her seferinde küçük bir düzeltme yapılır.
  - Sürekli sınama ile yapılır: değişikliklerin yeni hata oluşturmadığından emin olmak için her değişikliğin ardından kodun yeni hali sınanır. Bunun için birim sınamaları kullanılır; değiştirilen kısımlar ve onlarla bir adım ilişkili kısımların testleri yeniden çalıştırılır. Değişiklik kötüleştirdiyse hemen geri alınır; bunu sürüm kontrol yazılımı kolaylaştırır.
  - Ne zaman: kod derlenip toparlanmaya gerek duyduğunda; yeni özellik eklemekte zorlanılmaya başlandığında; bir hatayı bulmakta zorlanılıp bulununca da düzeltmek için birçok ayrı yeri kurcalamak gerektiğinde. Kısaca "kod kötü kokmaya başladığında".
- Code smell (kod kusuru)
  - İngilizcede "smelly" bir terslik olduğunu anlatır. "Hatalı kod" çevirisi doğru değildir, çünkü kod doğru çalışır, bug yoktur; ama esneklik ve bakım sorunlarına yol açar, hantallaşır, "yamalı bohçaya" döner. Hoca "kod kusuru" çevirisini tercih ediyor.
  - Kod kusurları genelde kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.
  - Öğrenmede ileriye ket vurma: eski bilgi ve alışkanlıkların yenisini öğrenmeyi zorlaştırması. Örnek: uzun süre yapısal programlama yaptıktan sonra nesneye yönelimde yapısal alışkanlıklarla devam etmek. "Elinde çekiç olan her şeyi çivi olarak görür."
  - Gereksinim değişiklikleri parça parça gelir; her biri için yapılan yerel düzeltmeler birikerek kodu genel en iyiden çok uzağa düşürebilir.
  - Bir refactoring eylemi bir yeri düzeltirken başka kod kusurları doğurabilir. Bu yüzden her eylemden sonra yeni kusur çıkıp çıkmadığına da bakılır ve gerekirse yeniden refactoring yapılır.
- Birim sınamaları (unit testing) (ekrandan, 00:40:00)
  - En küçük yazılım bileşeninin sınanmasıdır; nesneye yönelik programlamada bireysel sınıfların sınanmasıdır.
  - Ne zaman tasarlanır: kodlamadan önce (TDD, Test Driven Development/Design), kodlama sırasında ya da kodlamanın ardından.
  - TDD: iş mantığıyla ilgili tek satır kod yazılmadan sınamalar tasarlanır. Bir şey tek başına düzgün sınanamıyorsa tasarım baştan yanlıştır; bu, yaşam döngüsünün erken bir anında daha az masrafla düzeltme fırsatı verir. Hoca bunun herkes ve her problem için en iyi yol olmayabileceğini söyledi.
  - Şelale modelindeki gibi bütün kodlama bittikten sonra sınamak dezavantajlıdır: hata ne kadar geç bulunursa düzeltme maliyeti o kadar katlanarak artar. Çevik süreçte

her çevrimin sonunda o çevrimde yazılan kısımlar sınanır.
  - Kara kutu sınaması: bileşenin iç yapısını dikkate almadan, belirli girdi kümelerine karşılık doğru çıktı elde edilip edilmediğine bakılır; beyaz kutuya göre daha az çabayla verimli hata bulur. Beyaz kutu sınaması: iç yapı bilinerek denetim akışının her olası dalından geçecek şekilde daha fazla test vakası yazılır.
  - Bir sınıfın ihtiyaç duyduğu diğer sınıflar henüz yazılmamışsa onların yerine geçecek geçici kod yazılabilir: vekil, sahte, yalancı kod/sınıf (stub, dummy, surrogate, proxy). Vekil yalnız ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır. Hocanın örneği: karmaşık bir hesaptan sonra başarılı/başarısız döndüren metodun gövdesine geçici olarak `return true` yazmak.
- JUnit (ekrandan, 00:40:00, 00:50:00, 00:56:30)
  - Birim sınamaları için en yaygın araç ailesi xUnit'tir; x yerine dil ya da platformu simgeleyen harf gelir. Java için JUnit, C# için csUnit. Eclipse'te IDE ile hazır gelir (build path → add libraries); NetBeans'te de hazır gelir.
  - JUnit 3.x: her uygulama sınıfının (`AppClass1`) birim testi ayrı bir sınıfta (`TestAppClass1`) yazılır. Test sınıfının adı `Test` ile başlar ve `junit.framework.TestCase`'ten kalıtımla türetilir. Her `testXXX()` metodu bir test case'tir; parametre almaz; adı `test` ile başlar ve neyi sınadığını açıkça anlatır.
  - `TestCase`'in protected `setUp()` metodu override edilerek sınama öncesi "sahne" hazırlanır; `setUp()` her `testXXX()` öncesinde yeniden çalışır.
  - `Assert` sınıfının public static metotları: `assertEquals(beklenen, sonuc: Object)`, `assertNotEquals(Object, Object)`, `assertTrue(boolean)`, `assertFalse(boolean)`, `assertNull(Object)`, `assertNotNull(Object)`.
  - Test sınıfları `TestAll` gibi bir sınıfın statik `suite()` metodunda (dönüş tipi `Test`) bir test suite altında toplanabilir. `TestAll` yazmak zorunlu değildir; test sınıfları tek tek de çalıştırılabilir.
  - JUnit 4.x: geriye doğru uyumludur ve annotation desteği getirir (büyük/küçük harf duyarlı). Test sınıfları `TestCase`'ten türetilmek zorunda değildir; `import static org.junit.Assert.*;` ve `import org.junit.*;` eklenir. Metot adlarını `test` ile başlatmak gerekmez, `@Test` yeterlidir. `setUp` yerine `@Before` gelmiştir. `@Test(expected=SomeException.class)` ile beklenen bir istisnanın gerçekten atılıp atılmadığı sınanabilir; istisna olsa da olmasa da test akışı kesilmez.
  - Annotation'lar derleyicinin göz ardı ettiği, `@` ile başlayan bilgi satırlarıdır; Eclipse'teki `@SuppressWarnings` örneğinde olduğu gibi araçlara bilgi verir.
  - Hoca istisna sınaması gerekmedikçe 3.x sürümüyle devam ettiğini, dersteki örneklerin de 3.x olacağını söyledi.
- Fowler'ın video kiralama örneği (ekrandan, 01:20:00, 01:30:00, 01:32:20, 01:33:45, 01:35:20)
  - Refactoring bölümü Martin Fowler'ın "Refactoring" kitabından seçmelerle hazırlandı. Fowler önce bütüncül bir örnek verip ayrıntıya sonra iner; hoca önce kusurları ve refactoring eylemlerini anlatıp bütüncül örneği en sonda yapmayı tercih ediyor.
  - Sınıflar: `Customer` (`+statement()`), `Rental` (`-_daysRented: int`), `Movie` (`-_priceCode: int`). Bir müşterinin sıfır ya da daha fazla kira kaydı vardır; her kira kaydı tek bir filmle ilişkilidir. Kaynak kod: `refactoring/fowler00`.
  - `Rental` sınıfının gerekçesi: kiralama gün sayısı `Movie`'de tutulamaz (film başka müşterilere başka sürelerle kiralanır), `Customer`'da da tutulamaz (müşteri birden fazla film kiralar). "Müşteri film kiralar" cümlesindeki kiralama başlı başına bir varlıktır.
  - `statement()` makbuz basar. Sıra diyagramı: her kira için `getMovie()`, `getPriceCode()`, `getDaysRented()` ile tek kira için ücret, sonra bonus hesabı, sonra `getTitle()` ile makbuz satırı tamamlanır; döngü bitince biriken bilgiyle makbuz tamamlanır.
  - Fowler üye alan adlarını `_` ile başlatır; nedeni siyah beyaz kitapta küçük kod parçalarında üye alanı geçici değişkenden ayırt etmektir. Java'da bu bir tavsiye değildir.
  - Tarife (kod ekrandan): normal film (`REGULAR = 0`) 2.0, 2 günden sonraki her gün için +1.5; yeni film (`NEW_RELEASE = 1`) gün başına 3.0; çocuk filmi (`CHILDRENS = 2`) 1.5, 3 günden sonraki her gün için +1.5. Her kiralama 1 bonus puan; yeni film 1 günden fazla kiralanmışsa 1 puan daha.
  - Hocanın gördüğü ilk kusurlar: `statement()` çok uzun (long method); fiyat koduna göre `switch-case` var (kalıtım ve çok biçimlilik kullanılmalı); film türleri enum yerine `int` sabitlerle tanımlanmış ve geçersiz değer atanması engellenmiyor.
- JUnit ile örneğin sınanması (ekrandan, 01:42:30, 01:45:20, 01:48:05, 01:49:05, 01:51:10,

01:53:00)
  - Eclipse'te test sınıfları `testing` klasöründe, uygulama sınıfları `refactoring` klasöründe durur ama ikisi de aynı mantıksal paket (`fowler_00`) altında bildirilir; böylece paket düzeyindeki metotlara da erişilebilir.
  - `TestMovie.testMovie()`: üç türden birer film oluşturulur, `assertTrue`/`assertFalse` ile fiyat kodlarının birbirinden farklı olduğu sınanır. Üç assert bir metotta olduğu için tek test case sayılır.
  - `TestCustomer`: `setUp()` içinde müşteri ve dört film yalnız oluşturulur, kiralama ilişkileri her testte ayrıca kurulur. Testler: `testGetName`, `testStatementWhenEmpty`, `testStatementWithMoviesLongRent`, `testStatementWithMoviesShortRent`, `testNewReleaseRentalBonus`. Beklenen makbuz elle hesaplanıp `assertEquals(beklenen, sonuc)` ile karşılaştırılır.
  - Hata gösterimi: çocuk filminin başlangıç ücreti yanlışlıkla 1.0 yazılınca kırmızı çubuk çıktı, 5 testten 2'si düştü; JUnit beklenen ve elde edilen metinlerin farklılaştığı yeri köşeli parantezle gösterdi.
  - `TestAll.suite()`: `new TestSuite("Bütün testler")`, `addTestSuite(TestCustomer.class)`, `addTestSuite(TestMovie.class)`. `.class` ile sınıfın `Class` nesnesine erişilir (Reflection API).

Hocanın Özellikle Vurguladığı Kısımlar

- Mediator Observer değildir
  - Observer durum değişikliğinden haberdar olmaktır; Mediator ise eş bileşenler arasındaki M×N ilişkiyi merkezi bir noktaya toplar ve iş ya da eylem zinciri tetikletir. Hoca "o farkı hatırlayalım" dedi.
- Mediator bir ödünleşimdir
  - İlişki karmaşıklığı azalır ama merkezde karmaşık, yeniden kullanımı zor bir sınıf ve tek hata noktası oluşur. Kazanç ve kayıp tartılıp karar verilir.
- Refactoring dış davranışı değiştirmez
  - Değişen iç yapıdır; amaç iç kalite özellikleridir. Ad değiştirmek bile refactoring'dir.
- Refactoring küçük adımlarla ve birim sınamalarıyla yapılır
  - Bütün kodu silip baştan yazmak yerine küçük düzeltmeler yapılır; her adımdan sonra ilgili testler çalıştırılır, kötüleşme olursa geri alınır.
- Kod kusuru hata değildir
  - Kod çalışır ama esnekliği ve bakımı bozulmuştur. Bu yüzden "hatalı kod" değil "kusurlu kod" denir.
- Switch-case görmek tasarım uyarısıdır
  - Kendinizi switch-case ya da if-else kullanırken görüyorsanız, kalıtım ve çok biçimlilik kullanan daha iyi bir tasarım gerektiğinin önemli bir göstergesidir.
- Erken sınama
  - Hata ne kadar geç bulunursa düzeltme maliyeti o kadar katlanarak artar; bütün kodlamanın bitmesi beklenmez. Bağımlı sınıf henüz yoksa vekil kodla sınamaya devam edilir.

Kısa Tekrar Notları

- Mediator: nesne topluluğu arasındaki etkileşimi sarmalar; Colleague'ler yalnız Mediator ile konuşur.
- Mediator bileşenleri: `Mediator`, `ConcreteMediator`, `Colleague`, `ConcreteColleague`.
- Font Chooser örneği: `DialogDirector` (soyut `CreateWidgets()`, `WidgetChanged(Widget)`), `FontDialogDirector`, `Widget.Changed()` → `director->WidgetChanged(this)`.
- 5 nesne tam bağlı: 20 ilişki; Mediator ile (çift yönlü): 10 ilişki.
- Mediator dezavantajı: merkezde karmaşıklık, düşük yeniden kullanım, tek hata noktası.
- Refactoring: ne yaptığını değil nasıl yaptığını değiştirmek; iç kaliteyi artırmak.
- İç kalite geliştiriciyi, dış kalite kullanıcıyı ilgilendirir.
- Refactoring çevik, küçük artımlı ve sürekli sınamalı yapılır.
- Ne zaman: yeni özellik eklemek zorlaştığında, hata ayıklamak için birçok yeri kurcalamak gerektiğinde, "kod koktuğunda".
- Code smell = kod kusuru; çalışan ama kusurlu kod.

- Birim sınaması: NYP'de bireysel sınıfın sınanması; TDD, kodlama sırası ya da sonrası.
- Kara kutu: girdi-çıktı; beyaz kutu: iç akışın bütün dalları.
- Stub/dummy/surrogate/proxy: henüz yazılmamış sınıfların yerine geçici kod.
- JUnit 3.x: `TestCase`'ten türet, `testXXX()`, `setUp()`, `assertXXX`, `suite()`.
- JUnit 4.x: `@Test`, `@Before`, `@Test(expected=...)`, kalıtım gerekmez.
- Video kiralama: `Customer`–`Rental`–`Movie`; `statement()` uzun metot ve switch-case içerir.

Detaylı Açıklamalar

- Mediator'ın font penceresi örneğinde her widget yalnız kendi director'ını bilir. Bir widget değiştiğinde `Changed()` metodu director'a `WidgetChanged(this)` ile haber verir. Somut `FontDialogDirector` hangi widget'ın değiştiğine bakar, gerekli bilgiyi (örneğin `ListBox.GetSelection()`) alır ve gerekli bileşene iş yaptırır (örneğin `EntryField.SetText()`). Böylece widget'lar arasında doğrudan bağ kurulmaz. Soyut `DialogDirector`'daki `ShowDialog()` somut bir metottur ve soyut `CreateWidgets()`, `WidgetChanged()` adımlarını kullanır; bu Template Method kalıbının kullanımıdır.
- Mediator'da ilişki sayısının hesabı: n Colleague birbiriyle tam bağlıysa yönlü ilişki sayısı $n(n-1)$'dir; $n=5$ için 20. Mediator ile her Colleague'den Mediator'a ve Mediator'dan Colleague'e birer ilişki olur, toplam $2n = 10$. Ama bu sadeleşme merkezde toplanan karmaşıklıkla ödenir.
- Refactoring'de public arayüzlerin değişmemesine çalışılır; ama bir noktada saf refactoring yapılamayabilir ve dışa açık metotların ya da arayüzlerin değişmesi gerekebilir. Gerekiyorsa bu da yapılır.
- Birim sınamasında vekil kod kullanımı: sınanan sınıfın ihtiyaç duyduğu sınıfın gerçeklemesi bitmemişse, yalnız kendi projemizde o metodu örneğin `return true` dönecek şekilde yazarız. Böylece derleme hatasından kurtulur, o hesabın doğru olduğu durumlar için kendi sınıfımızı sınarız. Asıl kod yazıldığında birim sınamalarında hata çıkarsa o zaman düzeltilir; ama bunu beklemek pahasına şimdiden bulunabilecek hatalardan vazgeçilmez.
- JUnit 3.x'te bir test sınıfı yazmak için ek bir yapıcı yazmak ya da `super` yapıcısını çağırmak gerekmez; `TestCase`'ten türetmek ve `test` ile başlayan parametresiz metotlar eklemek yeter. Birden fazla test metodu aynı nesnelere ihtiyaç duyuyorsa bu nesneler üye alan yapılır ve `setUp()` içinde ilklendirilir. Hoca `setUp()`'ın her test metodundan önce çağrılmasının büyük olasılıkla Template Method kalıbıyla gerçeklendiğini söyledi.
- Fowler örneğinde `statement()` döngüsü şöyle işler (kod ekrandan, 01:32:20 ve 01:33:45):

```
Enumeration<Rental> rentals = _rentals.elements();
String result = "Rental Record for " + getName() + "\n";
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = rentals.nextElement();
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        break;
    }
    // add frequent renter points
    frequentRenterPoints ++;
    // add bonus for a two day new release rental
    // (devamı karede görünmüyor; hocanın anlatımına göre yeni film
```

```
    //  1 günden fazla kiralanmışsa bir puan daha eklenir)
    ...
}
```

- `testStatementWithMoviesLongRent` örneği (ekrandan, 01:48:05): The Matrix (normal) 3 gün, Monsters, Inc. (çocuk) 4 gün, Surrogates (yeni) 2 gün kiralanır. Hesap: Matrix $2.0 + (3-2) \times 1.5 = 3.5$; Monsters $1.5 + (4-3) \times 1.5 = 3.0$; Surrogates $2 \times 3.0 = 6.0$; toplam 12.5. Puan: üç kiralama için 3, Surrogates yeni film ve 1 günden fazla olduğu için +1, toplam 4. Beklenen makbuz `"Amount owed is 12.5"` ve `"You earned 4 frequent renter points"` satırlarıyla biter. Değerler python ile doğrulandı.
- Çocuk filmi başlangıç ücreti 1.5 yerine 1.0 yazılınca kısa kiralama testinde beklenen `Monsters, Inc. 1.5` ve `Amount owed is 6.5` yerine `1.0` ve `6.0` elde edildi (ekrandan, 01:51:10). JUnit'in farkı köşeli parantezle göstermesi hatanın çocuk filmi tarifesinde olduğunu hemen ortaya koydu; değer düzeltilince testler yeniden yeşile döndü.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.